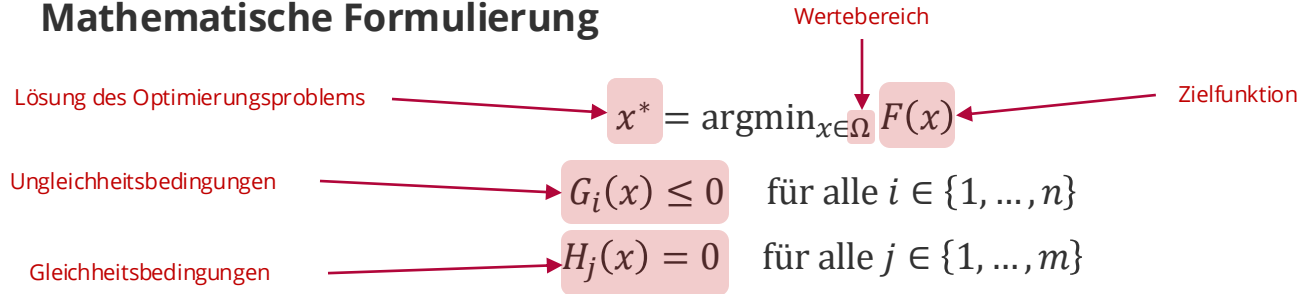


Programmiertechnik II

Optimierung

Ralf Herbrich

■ Mathematische Formulierung



■ Bemerkungen

- Wenn die Beschränkungen nicht erfüllbar sind, dann ist die Ausgabe die leere Menge. Das Problem ist nicht lösbar.
- Maximierung von $F(x)$ kann durch Minimierung von $-F(x)$ erreicht werden
- **In dieser Vorlesung:** Ω ausschließlich diskret
- Der Spezialfall $F(x) = \text{const}$ heißt *constraint satisfaction* Problem
- **Approximation:** $F(x_{\text{approx}}) \leq (1 + \varepsilon) \cdot F(x^*)$

Optimierungsprobleme: Beispiel

■ Wechselgeld ausgeben

□ Gegeben:

1. Feste Anzahl N_i von Münzen mit Nennwert W_i
2. Zielwert T der ausgegeben werden soll

□ Gesucht:

1. Kleinste Anzahl Münzen, um T auszugeben



$$x^* = \operatorname{argmin}_{x \in \Omega} F(x)$$

$$G_i(x) \leq 0 \quad \text{für alle } i \in \{1, \dots, n\}$$

$$H_j(x) = 0 \quad \text{für alle } j \in \{1, \dots, m\}$$

■ Darstellung als Optimierungsproblem

- x ist ein Vektor der Anzahl ausgegebener Münzen von Nennwert W_i

$$F(x) = \sum_i x_i \quad \leftarrow \text{Gesamtanzahl an Münzen}$$

$$G_i(x) = x_i - N_i \quad \leftarrow \text{Stellt sicher, dass nicht mehr Münzen ausgegeben werden, als vorhanden sind}$$

$$H(x) = \sum_i x_i \cdot W_i - T \quad \leftarrow \text{Der ausgegebene Betrag muss dem Zielwert entsprechen}$$

Programmiertechnik II

Unit 10 – Optimierung

1. *Greedy* Optimierung
2. Heuristische Optimierung
 - A* Algorithmus
3. Dynamische Programmierung
 - Optimale Matrixmultiplikation

1. **Greedy Optimierung**
2. Heuristische Optimierung
 - A* Algorithmus
3. Dynamische Programmierung
 - Optimale Matrixmultiplikation

■ Problemklasse:

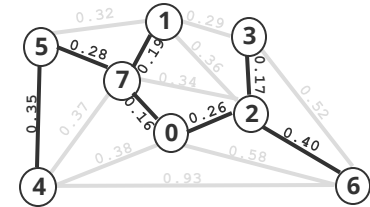
1. Es gibt eine **Nachbarschaftsrelation** $N(x) \subseteq \Omega$ im Raum Ω der Lösungen x , die es erlaubt Lösungen schrittweise aus anderen Lösungen aufzubauen
2. Die **Zielfunktion** F und Nebenbedingungen G_i und H_j sind effizient berechenbar für alle Nachbarn $N(x)$ einer Lösung x

■ Prinzip (*greedy* = „gierig“):

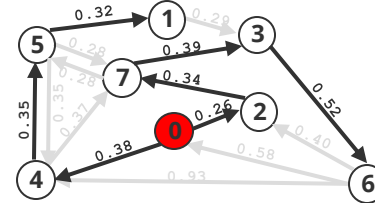
1. Starte bei einer Lösung x (welche eventuell G_i und H_j noch nicht erfüllt)
 - a) Wenn die Lösungen noch nicht G_i und H_j erfüllen, wähle $x' \in N(x)$ welches die Nebenbedingungen „am meisten“ erfüllen.
 - b) Wenn die Lösungen G_i und H_j erfüllen, wähle $x' \in N(x)$ welches die Nebenbedingungen erfüllt lässt aber die Zielfunktion F weiter minimiert.
2. Gehe zu 1. mit $x \leftarrow x'$ oder stoppe, wenn es keine bessere Lösung in $N(x)$ gibt

■ Beispiele:

- **Prim's Algorithmus:** x ist die Menge aller Kanten im minimalen Spannbaum (MST)
- **Dijkstra's Algorithmus:** x ist die Menge aller Kanten im Kürzeste-Pfade-Baum (SPT)



Minimaler Spannbaum



Kürzeste-Pfade-Baum von 0

Programmiertechnik II

Unit 10 – Optimierung

Beispiel: Wechselgeld ausgeben (ohne Ungleichheit)

- **Gegeben:** Ein Betrag T der gewechselt werden soll

$$\min_{\{x_1, \dots, x_8\}} \sum_{i=1}^8 x_i$$

Gesamtanzahl an Münzen

$$\text{so dass } H(x_1, \dots, x_8) = \sum_{i=1}^8 x_i \cdot W_i - T$$

Der ausgegebene Betrag muss dem Zielwert entsprechen



- **Greedy Algorithmus**

Solange $H(x_1, \dots, x_8) < 0$

Wähle $i = \operatorname{argmax}_{j \in \{1, \dots, 8\}, H(x_1, \dots, x_8) + W_j \leq 0} H(x_1, \dots, x_8) + W_j$

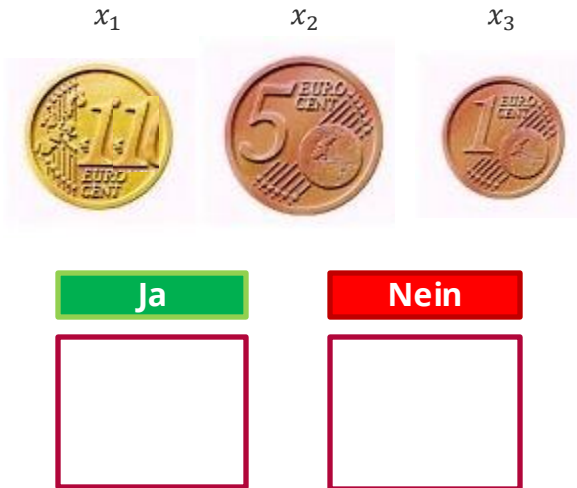
Erhöhe x_i um eins

- **Satz:** Der *Greedy Algorithmus* für Wechselgeld ist optimal!

- Wegen guter Auswahl der Münzwerte!

Beispiel: Wechselgeld ausgeben (Optimalität)

- **Frage:** Ist der *Greedy* Algorithmus auch noch optimal, wenn wir diese drei Münzen als Wechselgeld haben?



$$T = 14$$

$$T = 11 + 1 + 1 + 1$$

$$T = 15$$

$$T = 11 + 1 + 1 + 1 + 1 \neq 5 + 5 + 5$$

$$T = 26$$

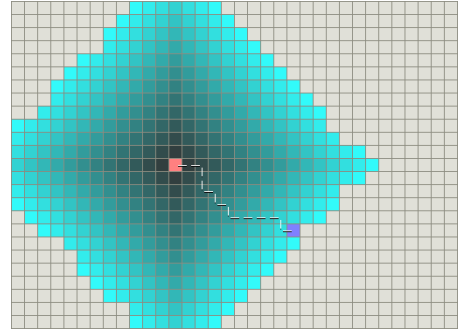
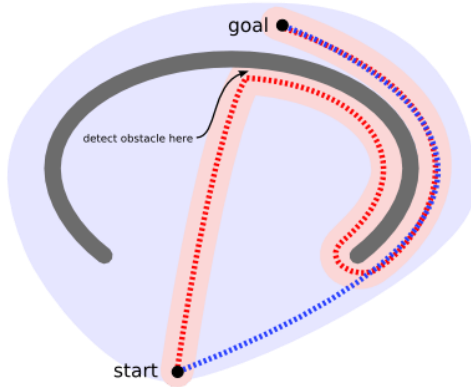
$$T = 11 + 11 + 1 + 1 + 1 + 1 \neq 11 + 5 + 5 + 5$$

- *Greedy* Verfahren führen zu **lokalen Optima**

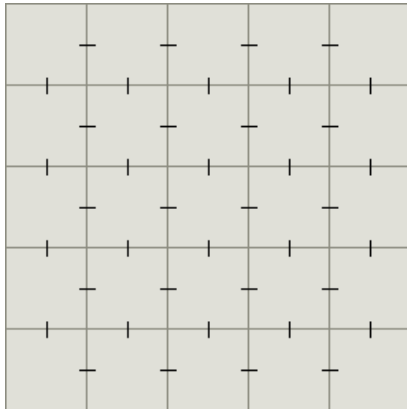
- Reicht manchmal aus
- Gute Laufzeit
- Entspricht manchmal globalem Optimum

1. *Greedy* Optimierung
2. **Heuristische Optimierung**
 - A* Algorithmus
3. Dynamische Programmierung
 - Optimale Matrixmultiplikation

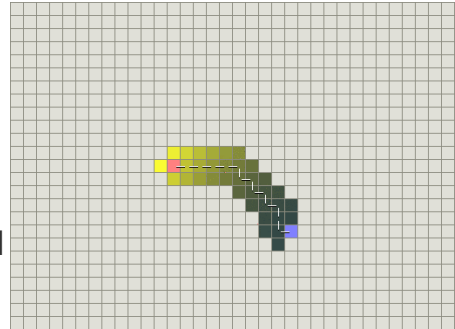
Pathfinding in Computerspielen



- **Dijkstra's Algorithmus** baut **greedy** den **SPT** auf.
- Wir könnten abbrechen, sobald der Zielknoten t erreicht werden.
 - Garantiert **Optimalität**
 - für viele Graphen gut genug, unter Bedingungen optimal
 - Langsam!
- **Greedy Best-first suche** versucht schnell eine **heuristische Lösung** zu finden.

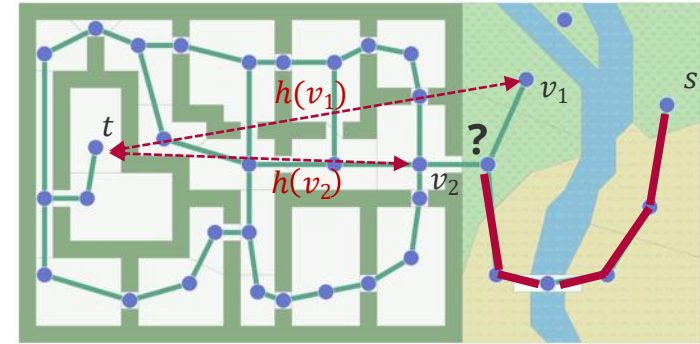


Graph **2D grid**
benachbarte Kacheln sind
durch Kanten verbunden



Greedy Best-First Suche

- **Idee 1:** Baue ausgehend vom Startknoten s sukzessive einen Baum auf über die Knoten, die am **nächsten zum Zielknoten t** sind, bis t enthalten ist.
 - Da die Nähe der Länge des kürzesten Pfades entspricht ist der beste Baum genau der kürzeste Pfad.
- **Problem:** Die Länge des kürzesten Pfades ist unbekannt!
- **Idee 2:** Schätze die Nähe zu t für jeden Knoten v basierend auf einer problemspezifischen **“Heuristik”** $h(v)$
 - **Beispiele:** Euklidische Distanz, Länge eines bekannten Pfades, ...
- Da die $h(v)$ nicht exakt ist, kann der Baum Abzweigungen haben (wir gehen zurück).



Greedy Best-First Suche

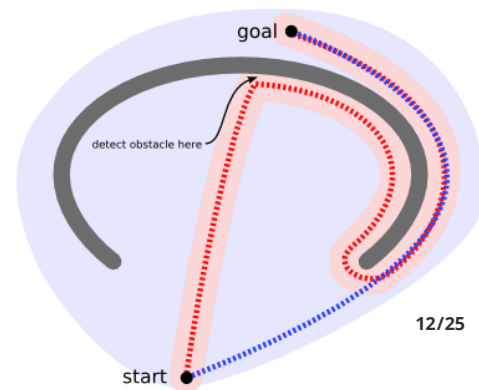
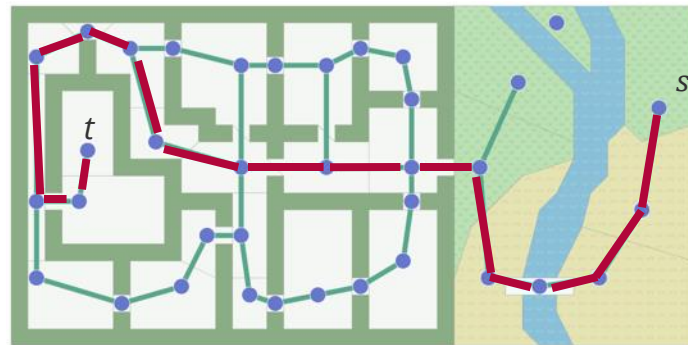
- **Algorithmus:** Ausgehend von s , baue sukzessive einen Baum auf über die Knoten v , die am **nächsten zum Zielknoten t** sind, bis t enthalten ist basierend auf **Heuristik $h(v)$** .

1. Beginne mit Knoten s im Baum
2. Wähle die Kante, die von einem Knoten im Baum zu einem Knoten w außerhalb des Baums zeigt, der nach $h(w)$ am nächsten an t ist.
3. Wiederhole Schritt 2 bis t im Baum ist.

- **Beobachtungen:**

1. Heuristik kann mit Glück schnell gute Lösungen finden
2. Kann schiefgehen und die Lösung **beliebig schlecht**.

- **Idee:** Kombiniere Heuristiken mit Dijkstra's Algorithmus



1. *Greedy* Optimierung
2. Heuristische Optimierung
 - **A* Algorithmus**
3. Dynamische Programmierung
 - Optimale Matrixmultiplikation

A* Algorithmus

- **Problem:** Gegeben ein gerichteter Graph $G = (V, E)$ und zwei Knoten $s \in V$ und $t \in V$, finde einen gerichteten Pfad von s zu t so dass kein anderer Pfad von s zu t ein geringeres Gewicht hat.
 - Zusätzlich haben wir eine **Heuristik** $h(v)$ die eine Zahl berechnet, die nie größer ist, als die Länge des kürzesten Pfades von $v \in V$ zum Ziel $t \in V$!
- **A* Algorithmus:** Erweiterung von Dijkstra's Algorithmus, bei dem die Heuristik benutzt wird!
 - **Dijkstra:** Es werden die kürzesten Pfade von s zu allen anderen Knoten berechnet (SPT), indem alle Nachbarn des Knoten, der **am nächsten an s ist**, relaxiert
 - **A* Algorithmus:** Es wird der kürzeste Pfad von s zu t berechnet, indem alle Nachbarn des Knoten, der geschätzt **am nächsten an t ist**, relaxiert
 - Beide Algorithmen benutzen eine Prioritätswarteschlange, um den nächsten Knoten zu verwalten.



Peter E. Hart
(1941 –)



Bertram Raphael
(1936 –)



Nils Nilsson
(1933 – 2019)

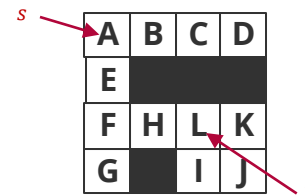
A* Algorithmus

- **Idee:** Expandiere immer den Knoten der am nächsten am Zielknoten ist

1. Initialisiere alle kürzesten Distanzen $d(v)$ vom Startknoten s zu alle Knoten $v \neq s$ mit ∞ und $d(s) = 0$
2. Füge s mit der Priorität $d(s) + h(s)$ in die Warteschlange ein
3. Entferne den nächsten Knoten v aus der Prioritätswarteschlange
4. Wenn v der Zielknoten t ist, gebe den kürzesten Pfad zurück
5. Ansonsten relaxiere alle Kanten $e = (v, w) \in E$:
Wenn $d(v) + \text{weight}(e) \leq d(w)$ dann
 - $d(w) \leftarrow d(v) + \text{weight}(e)$
 - Füge w mit der Priorität $d(w) + h(w)$ in die Warteschlange ein
6. Wiederhole Schritt 3, 4 und 5

(A, 4)
(B, 4) (E, 4)
(E, 4) (C, 4)
(C, 4) (F, 4)
(F, 4) (D, 6)
(H, 4) (D, 6) (G, 6)

v	$h(v)$	$d_0(v)$	$d_1(v)$	$d_2(v)$	$d_3(v)$	$d_4(v)$	$d_5(v)$
A	4	0	0	0	0	0	0
B	3	∞	1	1	1	1	1
C	2	∞	∞	2	2	2	2
D	3	∞	∞	∞	∞	3	3
E	3	∞	1	1	1	1	1
F	2	∞	∞	∞	2	2	2
G	3	∞	∞	∞	∞	∞	3
H	1	∞	∞	∞	∞	∞	3
I	1	∞	∞	∞	∞	∞	∞
J	2	∞	∞	∞	∞	∞	∞
K	1	∞	∞	∞	∞	∞	∞
L	0	∞	∞	∞	∞	∞	∞



Programmiertechnik II

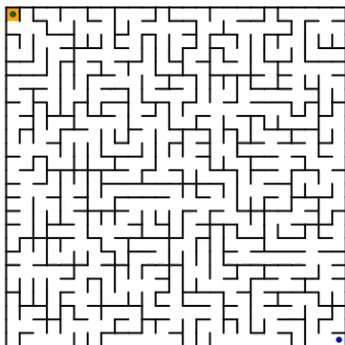
Unit 10 – Optimierung

A* Algorithmus: Heuristik

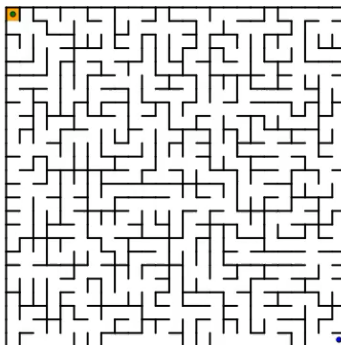
■ **Beobachtung:** Je nach Qualität der Heuristik gilt

- $h(v) = 0$: Dijkstra's Algorithmus
- $h(v) = \text{Länge des kürzesten Pfads bis zum Zielknoten } t$: Optimaler Pfad in n Schritten
- Je näher $h(v)$ an der Länge des kürzesten Pfads zu t ist, umso weniger Knoten werden exploriert

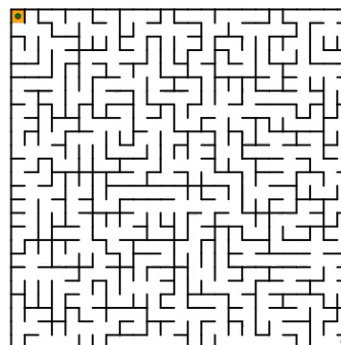
$$h(v) = 0$$



$$h(v) = \|\vec{v} - \vec{t}\|_2$$



$$h(v) = \|\vec{v} - \vec{t}\|_1$$



- Wenn gilt $h(v) \leq \text{weight}(v, w) + h(w)$ dann wird jeder Knoten maximal **einmal** besucht!

1. *Greedy* Optimierung
2. Heuristische Optimierung
 - A* Algorithmus
3. **Dynamische Programmierung**
 - Optimale Matrixmultiplikation

- **Beobachtung:** Oft hat ein Optimierungsproblem eine (rekursive) Struktur, bei der es aus der Kombination von n Optimierungsproblemen berechenbar ist.

- **Beispiel:** Fibonacci Zahlen

$$\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$$

- **Idee:** Dynamische Programmierung

1. Berechne die optimale Lösung für die kleinsten Teilprobleme und speichere (*memorize*) diese.
2. Benutze die gespeicherten Ergebnisse, um das nächstgrößere Teilproblem zu lösen.

- **Beispiel:** Fibonacci Zahlen; Teilprobleme über n mit $\text{fib}(0) = 0$ und $\text{fib}(1) = 1$



Richard E. Bellmann
(1920 – 1984)



Donald Michie
(1923 – 2007)

```
// Fibonacci numbers without memoization
long long fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}
```

```
// Fibonacci numbers with memoization
long long fib_fast(int n, long long* memo) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    if (memo[n] == -1)
        memo[n] = fib_fast(n - 1, memo) + fib_fast(n - 2, memo);
    return memo[n];
}
```

```
➔ unit10 git:(main) x time ./fib 47
fib(47) = 2971215073
./fib 47 9.64s user 0.00s system 99% cpu 9.677 total
➔ unit10 git:(main) x time ./fib_fast 47
fib(47) = 2971215073
./fib_fast 47 0.00s user 0.00s system 50% cpu 0.006 total
```

- Das Prinzip der dynamischen Programmierung basiert auf **Bellmann's sequentiellen Entscheidungsproblem:**

1. Wir haben ein System im Urzustand s_0 (z.B., Position in einem Labyrinth)
2. Wir haben in jedem Zeitschritt die Möglichkeit eine von k Aktionen a_t auszuführen (z.B., nach vorn/rechts/links/hinten gehen)
3. Dafür bekommen wir eine Belohnung $r(s_t, a_t)$ (z.B., eine Strafe von -1 wenn wir noch nicht am Ziel sind)
4. Es gibt eine Dynamik, die den Zustand s_t aufgrund der Aktion a_t verändert, $s_{t+1} = D(s_t, a_t)$

- **Ziel:** Finde die Aktionen $a_0^*, \dots, a_{T-1}^*$, welche die Summe $\sum_{t=0}^{T-1} r(s_t, a_t)$ maximiert

- **Bellmann Gleichung:** Wenn wir die Wertefunktion V_i definieren

$$V_i(s_i) = \max_{a_i, \dots, a_{T-1}} \sum_{t=i}^{T-1} r(s_t, a_t)$$

dann gilt

$$V_i(s_i) = \max_{a_i} [r(s_i, a_i) + V_{i+1}(D(s_i, a_i))]$$

Optimierung über k^T Werte

Programmiertechnik II

Unit 10 – Optimierung

Berechnung und Speicherung
von Teilproblem V_{i+1}

Optimierung über k Werte



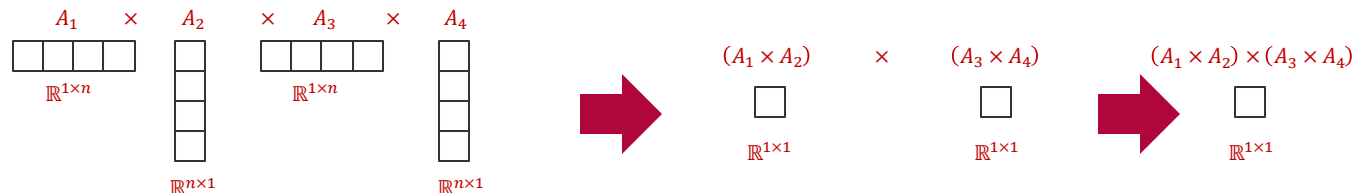
Richard E. Bellmann
(1920 – 1984)

1. *Greedy* Optimierung
2. Heuristische Optimierung
 - A* Algorithmus
3. Dynamische Programmierung
 - **Optimale Matrixmultiplikation**

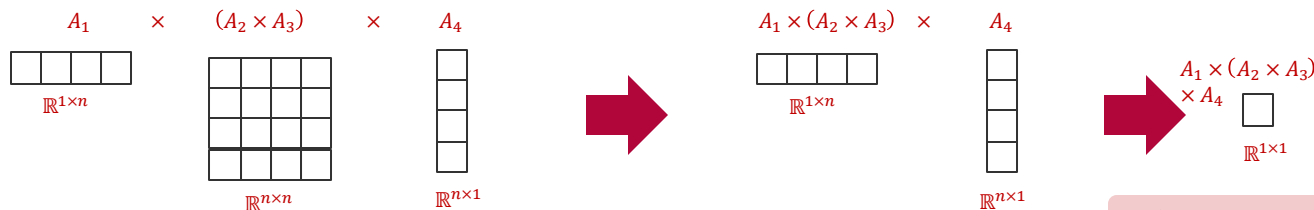
Matrixmultiplikation

- **Problem:** Wenn wir mehr als zwei Matrizen multiplizieren, gibt es mehrere Möglichkeiten die Multiplikation durchzuführen, welche deutlich unterschiedlichen Aufwand haben können!

□ **Beispiel:**



$2n + 1$ multiplications



$n^2 + 2n$ multiplications

Programmiertechnik II

Unit 10 – Optimierung

Matrixmultiplikation mit Dynamischer Programmierung

- **Idee:** Gegeben die Matrizen $A_1, \dots, A_n$, berechnen wir die minimale Anzahl $m_{i,j}$ von Multiplikationen um die Matrizen $A_i, A_{i+1}, \dots, A_j$ zu multiplizieren!
- **Bellmann Gleichung**

$$m_{i,j} = \min_{k=i, \dots, j-1} m_{i,k} + m_{k+1,j} + \text{rows}(A_i) \cdot \text{columns}(A_k) \cdot \text{columns}(A_j)$$

Minimale Multiplikationen um $A_i \times \dots \times A_k$ auszurechnen

Minimale Multiplikationen um $A_{k+1} \times \dots \times A_j$ auszurechnen

Multiplikationen um Matrixprodukt aus $(A_i \times \dots \times A_k)$ und $(A_{k+1} \times \dots \times A_j)$ auszurechnen

$$s_{i,j} = \arg \min_{k=i, \dots, j-1} m_{i,k} + m_{k+1,j} + \text{rows}(A_i) \cdot \text{columns}(A_k) \cdot \text{columns}(A_j)$$

Speichert den Index an dem die Multiplikation $A_i \times \dots \times A_j$ in zwei Produkte zerlegt wird, um die Anzahl der Multiplikationen zu minimieren

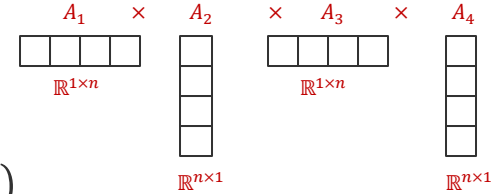
- **Initialisierung** mit $m_{i,i} = 0$
- **Iteration** über die Länge von $j - i$

Optimale Matrixmultiplikation: Beispiel

■ Bellmann Gleichungen

$$m_{i,j} = \min_{k=i,\dots,j-1} m_{i,k} + m_{k+1,j} + \text{rows}(A_i) \cdot \text{columns}(A_k) \cdot \text{columns}(A_j)$$

$$s_{i,j} = \arg \min_{k=i,\dots,j-1} m_{i,k} + m_{k+1,j} + \text{rows}(A_i) \cdot \text{columns}(A_k) \cdot \text{columns}(A_j)$$



■ Algorithmus

$$m_{1,2} = m_{1,1} + m_{2,2} + \text{rows}(A_1) \cdot \text{columns}(A_1) \cdot \text{columns}(A_2)$$

$$m_{2,3} = m_{2,2} + m_{3,3} + \text{rows}(A_2) \cdot \text{columns}(A_2) \cdot \text{columns}(A_3)$$

$$m_{3,4} = m_{3,3} + m_{4,4} + \text{rows}(A_3) \cdot \text{columns}(A_3) \cdot \text{columns}(A_4)$$

$$m_{1,3} = \min_{k \in \{1,2\}} m_{1,k} + m_{k+1,3} + \text{rows}(A_1) \cdot \text{columns}(A_k) \cdot \text{columns}(A_3)$$

$$m_{2,4} = \min_{k \in \{2,3\}} m_{2,k} + m_{k+1,4} + \text{rows}(A_2) \cdot \text{columns}(A_k) \cdot \text{columns}(A_4)$$

$$m_{1,4} = \min_{k \in \{1,2,3\}} m_{1,k} + m_{k+1,4} + \text{rows}(A_1) \cdot \text{columns}(A_k) \cdot \text{columns}(A_4)$$

$m_{i,j}$	1	2	3	4
1	0	n	$2n$	$2n+1$
2	0	0	n^2	$2n$
3	0	0	0	n
4	0	0	0	0

$s_{i,j}$	1	2	3	4
1		1	2	2
2			2	2
3				3
4				

Programmiertechnik II

Unit 10 – Optimierung

■ **Greedy Optimierung**

- Wenn noch keine optimale Lösung gefunden ist, evaluiere die „Nachbarschaft“-Lösung, die so „nah“ als möglich am Ziel ist
- Dijkstra's und Prim's Algorithmus sind Beispiele von *greedy* Algorithmen

■ **Heuristische Optimierung und A* Algorithmus**

- Eine Heuristik ist eine Abschätzung der Kosten bis zur optimalen Lösung sind
- Wenn die Heuristik Null ist, gibt es keine Verbesserungen
- Wenn die Heuristik die echten verbleibenden Kosten sind, wird die optimale Lösung sofort durch A* gefunden!

■ **Dynamische Programmierung**

- Paradigma für Algorithmen, dass Rechenzeit durch Speichern von Zwischenlösung einspart
- Kann immer dann genutzt werden, wenn ein Problem in „kleinere“ Teilprobleme zerlegt werden kann und die optimale Entscheidung durch optimale Lösung der Teilprobleme berechnet werden kann („Bellmann Gleichung“)

Viel Spaß bis zur Klausur!